

Algorithm Analysis-I

Dr.Kumkum Saxena

Order Analysis

- Judging the Efficiency/Speed of an Algorithm
 - Thus far, we've looked at a few different algorithms:
 - Max # of 1's
 - Linear Search vs Binary Search
 - Sorted List Matching Problem
 - and others
 - But we haven't really examined them, in detail, regarding their efficiency or speed

Order Analysis

- Judging the Efficiency/Speed of an Algorithm
 - We will use Order Notation to approximate two things about algorithms:
 - 1) **How much time they take**
 - 2) How much memory (space) they use
 - Note:
 - It is nearly impossible to figure out the exact amount of time an algorithm will take
 - Each algorithm gets translated into smaller and smaller machine instructions
 - Each of these instructions take various amounts of time to execute on different computers

Order Analysis

- Judging the Efficiency/Speed of an Algorithm
 - Note:
 - Also, we want to judge algorithms independent of their implementation
 - Thus, rather than figure out an algorithm's exact running time
 - We only want an approximation (Big-O approximation)
 - Assumptions: we assume that each statement and each comparison in C takes some constant amount of time
 - Also, most algorithms have some type of input
 - With sorting, for example, the size of the input (typically referred to as n) is the number of numbers to be sorted
 - Time and space used by an algorithm function of the input

Big-O Notation

- What is Big O?

- Big O comes from Big-O Notation

- In C.S., we want to know how efficient an algorithm is...how “fast” it is
 - More specifically...we want to know **how the performance of an algorithm responds to changes in problem size**

Big-O Notation

■ What is Big O?

- The goal is to provide a qualitative insight on the # of operations for a problem size of n elements.
- And this total # of operations can be described with a mathematical expression in terms of n .
 - This expression is known as Big-O
- The **Big-O** notation is a way of measuring the order of magnitude of a mathematical expression.
- $O(n)$ means “of the order of n ”

Big-O Notation

- Consider the expression:

- $f(n) = 4n^2 + 3n + 10$

- How fast is this “growing”?

- There are three terms:

- the $4n^2$, the $3n$, and the 10

- As n gets bigger, which term makes it get larger fastest?

- Let's look at some values of n and see what happens?

n	$4n^2$	$3n$	10
1	4	3	10
10	400	30	10
100	40,000	300	10
1000	4,000,000	3,000	10
10,000	400,000,000	30,000	10
100,000	40,000,000,000	300,000	10
1,000,000	4,000,000,000,000	3,000,000	10

Big-O Notation

- Consider the expression:

- $f(n) = 4n^2 + 3n + 10$

- How fast is this “growing”?

- Which term makes it get larger fastest?

- As n gets larger and larger, the $4n^2$ term DOMINATES the resulting answer

- $f(1,000,000) = 4,000,003,000,010$

- The idea of behind Big-O is to reduce the expression so that it captures the qualitative behavior in the simplest terms.

Big-O Notation

- Consider the expression: $f(n) = 4n^2 + 3n + 10$
 - How fast is this “growing”?
 - Look at VERY large values of n
 - **eliminate** any term whose contribution to the total ceases to be significant as n get larger and larger
 - of course, this also includes constants, as they little to no effect with larger values of n
 - Including constant factors (coefficients)
 - So we ignore the constant 10
 - And we can also ignore the 3n
 - Finally, we can eliminate the constant factor, 4, in front of n^2
 - We can approximate the order of this function, $f(n)$, **as n^2**
 - We can say, **$O(4n^2 + 3n + 10) = O(n^2)$**
 - In conclusion, we say that $f(n)$ takes $O(n^2)$ steps to execute

Big-O Notation

- Some basic examples:
 - What is the Big-O of the following functions:
 - $f(n) = 4n^2 + 3n + 10$
 - Answer: $O(n^2)$
 - $f(n) = 76,756,234n^2 + 427,913n + 7$
 - Answer: $O(n^2)$
 - $f(n) = 74n^8 - 62n^5 - 71562n^3 + 3n^2 - 5$
 - Answer: $O(n^8)$
 - $f(n) = 42n^4 \cdot (12n^6 - 73n^2 + 11)$
 - Answer: $O(n^{10})$
 - $f(n) = 75n \cdot \log n - 415$
 - Answer: $O(n \cdot \log n)$

Big-O Notation

- Consider the expression: $f(n) = 4n^2 + 3n + 10$
 - How fast is this “growing”?
 - We can say, $O(4n^2 + 3n + 10) = O(n^2)$
 - Till now, we have one function:
 - $f(n) = 4n^2 + 3n + 10$
 - Let us make a second function, **$g(n)$**
 - It's just a letter right? We could have called it $r(n)$ or $x(n)$
 - Don't get scared about this
 - Now, let $g(n)$ equal n^2
 - **$g(n) = n^2$**
 - So now we have two functions: **$f(n)$** and **$g(n)$**
 - We said (above) that $O(4n^2 + 3n + 10) = O(n^2)$
 - Similarly, we can say that the **order of $f(n)$ is $O[g(n)]$** .

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- Think about the two functions we just had:
 - $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$
 - We agreed that $O(4n^2 + 3n + 10) = O(n^2)$
 - Which means we agreed that the order of **$f(n)$ is $O(g(n))$**
- **That's all this definition says!!!**
- $f(n)$ is big-O of $g(n)$, if there is a c ,
 - (c is a constant)
- such that $f(n)$ is not larger than $c \cdot g(n)$ for sufficiently large values of n (greater than N)

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- Think about the two functions we just had:
 - $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$
- f is big-O of g , if there is a c such that f is not larger than $c \cdot g$ for sufficiently large values of n (greater than N)
 - So given the two functions above, does there exist some constant, c , that would make the following statement true?
 - $f(n) \leq c \cdot g(n)$
 - $4n^2 + 3n + 10 \leq c \cdot n^2$
 - If there does exist this c , then $f(n)$ is $O(g(n))$
- Let's go see if we can come up with the constant, c

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- PROBLEM: Given our two functions,
 - $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$
- **Find the c such that $4n^2 + 3n + 10 \leq c \cdot n^2$**
- Clearly, c cannot be 4 or less
 - Cause even if it was 4, we would have:
 - $4n^2 + 3n + 10 \leq 4n^2$
 - This is NEVER true for any positive value of n !
 - So **c must be greater than 4**
- Let us try with c being equal to 5
 - $4n^2 + 3n + 10 \leq 5n^2$

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- PROBLEM: Given our two functions,

- $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$

- **Find the c** such that $4n^2 + 3n + 10 \leq c \cdot n^2$

- $4n^2 + 3n + 10 \leq 5n^2$

- For what values of n , if ANY at all, is this true?

n	$4n^2 + 3n + 10$	$5n^2$
1	$4(1) + 3(1) + 10 = 17$	$5(1) = 5$

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- PROBLEM: Given our two functions,

- $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$

- **Find the c** such that $4n^2 + 3n + 10 \leq c \cdot n^2$

- $4n^2 + 3n + 10 \leq 5n^2$

- For what values of n , if ANY at all, is this true?

n	$4n^2 + 3n + 10$	$5n^2$
1	$4(1) + 3(1) + 10 = 17$	$5(1) = 5$
2	$4(4) + 3(2) + 10 = 32$	$5(4) = 20$

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- PROBLEM: Given our two functions,

- $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$

- **Find the c** such that $4n^2 + 3n + 10 \leq c \cdot n^2$

- $4n^2 + 3n + 10 \leq 5n^2$

- For what values of n , if ANY at all, is this true?

n	$4n^2 + 3n + 10$	$5n^2$
1	$4(1) + 3(1) + 10 = 17$	$5(1) = 5$
2	$4(4) + 3(2) + 10 = 32$	$5(4) = 20$
3	$4(9) + 3(3) + 10 = 55$	$5(9) = 45$

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- PROBLEM: Given our two functions,

- $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$

- **Find the c** such that $4n^2 + 3n + 10 \leq c \cdot n^2$

- $4n^2 + 3n + 10 \leq 5n^2$

- For what values of n , if ANY at all, is this true?

n	$4n^2 + 3n + 10$	$5n^2$
1	$4(1) + 3(1) + 10 = 17$	$5(1) = 5$
2	$4(4) + 3(2) + 10 = 32$	$5(4) = 20$
3	$4(9) + 3(3) + 10 = 55$	$5(9) = 45$
4	$4(16) + 3(4) + 10 = 86$	$5(16) = 80$

But now let's try
larger values of n .

- For $n = 1$ through 4, this statement is NOT true

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- PROBLEM: Given our two functions,

- $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$

- **Find the c** such that $4n^2 + 3n + 10 \leq c \cdot n^2$

- $4n^2 + 3n + 10 \leq 5n^2$

- For what values of n , if ANY at all, is this true?

n	$4n^2 + 3n + 10$	$5n^2$
1	$4(1) + 3(1) + 10 = 17$	$5(1) = 5$
2	$4(4) + 3(2) + 10 = 32$	$5(4) = 20$
3	$4(9) + 3(3) + 10 = 55$	$5(9) = 45$
4	$4(16) + 3(4) + 10 = 86$	$5(16) = 80$
5	$4(25) + 3(5) + 10 = 125$	$5(25) = 125$

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- PROBLEM: Given our two functions,

- $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$

- **Find the c** such that $4n^2 + 3n + 10 \leq c \cdot n^2$

- $4n^2 + 3n + 10 \leq 5n^2$

- For what values of n , if ANY at all, is this true?

n	$4n^2 + 3n + 10$	$5n^2$
1	$4(1) + 3(1) + 10 = 17$	$5(1) = 5$
2	$4(4) + 3(2) + 10 = 32$	$5(4) = 20$
3	$4(9) + 3(3) + 10 = 55$	$5(9) = 45$
4	$4(16) + 3(4) + 10 = 86$	$5(16) = 80$
5	$4(25) + 3(5) + 10 = 125$	$5(25) = 125$
6	$4(36) + 3(6) + 10 = 172$	$5(36) = 180$

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***

- PROBLEM: Given our two functions,

- $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$

- **Find the c** such that $4n^2 + 3n + 10 \leq c \cdot n^2$

- $4n^2 + 3n + 10 \leq 5n^2$

- For what values of n , if ANY at all, is this true?

- So when $n = 5$, the statement finally becomes true

- And when $n > 5$, it remains true!

- So our constant, 5, works for all $n \geq 5$.

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $\underline{f(n) \leq c \cdot g(n)}$ for all $n \geq N$.***

- PROBLEM: Given our two functions,
 - $f(n) = 4n^2 + 3n + 10$, and $g(n) = n^2$
- **Find the c** such that $4n^2 + 3n + 10 \leq c \cdot n^2$
- So our constant, 5, works for all $n \geq 5$.
- Therefore, **$f(n)$ is $O(g(n))$** per our definition!
- Why?
- Because **there exists positive integers, c and N ,**
 - Just so happens in this case that $c = 5$ and $N = 5$
- **such that $f(n) \leq c \cdot g(n)$.**

Big-O Notation

■ Definition:

- ***$f(n)$ is $O[g(n)]$ if there exists positive integers c and N , such that $f(n) \leq c \cdot g(n)$ for all $n \geq N$.***
 - What we can gather is that:
 - **$c \cdot g(n)$ is an upper bound on the value of $f(n)$.**
 - It represents the worst possible scenario of running time.
 - The number of operations is, at worst, proportional to $g(n)$ for all large values of n .

Big-O Notation

- Summing up the basic properties for determining the order of a function:
 - 1) If you've got multiple functions added together, the fastest growing one determines the order
 - 2) Multiplicative constants don't affect the order
 - 3) If you've got multiple functions multiplied together, the overall order is their individual orders multiplied together

Big-O Notation

- Some basic examples:
 - What is the Big-O of the following functions:
 - $f(n) = 4n^2 + 3n + 10$
 - Answer: $O(n^2)$
 - $f(n) = 76,756,234n^2 + 427,913n + 7$
 - Answer: $O(n^2)$
 - $f(n) = 74n^8 - 62n^5 - 71562n^3 + 3n^2 - 5$
 - Answer: $O(n^8)$
 - $f(n) = 42n^4 \cdot (12n^6 - 73n^2 + 11)$
 - Answer: $O(n^{10})$
 - $f(n) = 75n \cdot \log n - 415$
 - Answer: $O(n \cdot \log n)$

Big-O Notation

- Quick Example of Analyzing Code:
 - Use big-O notation to analyze the time complexity of the following fragment of C code:

```
for (k=1; k<=n/2; k++) {  
    sum = sum + 5;  
}  
  
for (j = 1; j <= n*n; j++) {  
    delta = delta + 1;  
}
```

Big-O Notation

■ Quick Example of Analyzing Code:

■ So look at what's going on in the code:

- We care about the total number of REPETITIVE operations.

- Remember, we said we care about the running time for LARGE values of n
- So in a **for loop**, with n as part of the comparison value determining when to stop

```
for (k=1; k<=n/2; k++)
```
- Whatever is INSIDE that loop will be executed a LOT of times
- So we examine the code within this loop and see how many operations we find
 - When we say operations, we're referring to mathematical operations such as $+$, $-$, $*$, $/$, etc.

Big-O Notation

- Quick Example of Analyzing Code:
 - So look at what's going on in the code:
 - The number of operations executed by these loops is the sum of the individual loop operations.
 - We have 2 loops,

```
for (k=1; k<=n/2; k++) {  
    sum = sum + 5;  
}  
  
for (j = 1; j <= n*n; j++) {  
    delta = delta + 1;  
}
```

Big-O Notation

- Quick Example of Analyzing Code:
 - So look at what's going on in the code:
 - The number of operations executed by these loops is the sum of the individual loop operations.
 - We have 2 loops,
 - The first loop runs $n/2$ times
 - Each iteration of the first loop results in one operation
 - The + operation in: `sum = sum + 5;`
 - So there are $n/2$ operations in the first loop
 - The second loop runs n^2 times
 - Each iteration of the second loop results in one operation
 - The + operation in: `delta = delta + 1;`
 - So there are n^2 operations in the second loop.

Big-O Notation

- Quick Example of Analyzing Code:
 - So look at what's going on in the code:
 - The number of operations executed by these loops is the sum of the individual loop operations.
 - The first loop has $n/2$ operations
 - The second loop has n^2 operations
 - They are NOT nested loops.
 - One loop executes AFTER the other completely finishes
 - So we simply ADD their operations
 - The total number of operations would be $n/2 + n^2$
 - In Big-O terms, we can express the number of operations as $O(n^2)$

Big-O Notation

- Common orders (listed from slowest to fastest growth)

Function	Name
1	Constant
$\log n$	Logarithmic
n	Linear
$n \log n$	Poly-log
n^2	Quadratic
n^3	Cubic
2^n	Exponential
$n!$	Factorial

Big-O Notation

- **O(1)** or “Order One”: **Constant time**
 - does not mean that it takes only one operation
 - does mean that the work doesn't change as n changes
 - is a notation for “constant work”
 - An example would be finding the smallest element in a sorted array
 - There's nothing to search for here
 - The smallest element is always at the beginning of a sorted array
 - So this would take $O(1)$ time

Big-O Notation

- **O(n)** or “Order n”: **Linear time**

- does not mean that it takes n operations
 - maybe it takes $3*n$ operations, or perhaps $7*n$ operations
- does mean that the **work changes in a way that is proportional to n**
- Example:
 - If the input size doubles, the running time also doubles
- is a notation for “**work grows at a linear rate**”
- You usually can’t really do a lot better than this for most problems we deal with
 - After all, you need to at least examine all the data right?

Big-O Notation

- $O(n^2)$ or “Order n^2 ”: Quadratic time
 - If input size doubles, running time increases by a factor of 4
- $O(n^3)$ or “Order n^3 ”: Cubic time
 - If input size doubles, running time increases by a factor of 8
- $O(n^k)$: Other polynomial time
 - Should really try to avoid high order polynomial running times
 - However, it is considered good from a theoretical standpoint

Big-O Notation

- **$O(2^n)$** or “Order 2^n ”: **Exponential time**
 - more theoretical rather than practical interest because they cannot reasonably run on typical computers for even for moderate values of n .
 - Input sizes bigger than 40 or 50 become unmanageable
 - Even on faster computers
- **$O(n!)$** : even worse than exponential!
 - Input sizes bigger than 10 will take a long time

Big-O Notation

- **$O(n \log_2 n)$** :

- Only slightly worse than $O(n)$ time
 - And $O(n \log_2 n)$ will be much less than $O(n^2)$
 - This is the running time for the better sorting algorithms we will go over (later)

- **$O(\log_2 n)$** or “Order $\log_2 n$ ”: **Logarithmic time**

- If input size doubles, running time increases ONLY by a constant amount
- **any algorithm that halves the data** remaining to be processed on each iteration of a loop will be an **$O(\log_2 n)$ algorithm**.

Big-O Notation – Practical Problems

- Practical Problems that can be solved utilizing order notation:
 - Example:
 - You are told that algorithm A runs in $O(n)$ time
 - You are also told the following:
 - For an input size of 10
 - The algorithm runs in 2 milliseconds
 - As a result, you can expect that for an input size of 500, the algorithm would run in 100 milliseconds!
 - Notice the input size jumped by a multiple of 50
 - From 10 to 500
 - Therefore, given a $O(n)$ algorithm, the running time should also jump by a multiple of 50, **which it does!**

Big-O Notation – Practical Problems

- Practical Problems that can be solved utilizing order notation:
 - General process of solving these problems:
 - We know that Big-O is NOT exact
 - It's an upper bound on the actual running time
 - So when we say that an **algorithm runs in $O(f(n))$ time**,
 - **Assume** the EXACT **running time is $c \cdot f(n)$**
 - where c is some constant
 - Using this assumption,
 - we can use the information in the problem to solve for c
 - Then we can use this c to answer the question being asked
 - Examples will clarify...

Big-O Notation – Practical Problems

- Practical Problems that can be solved utilizing order notation:
 - Example 1: Algorithm A runs in **$O(n^2)$** time
 - For an input size of 4, the running time is 10 milliseconds
 - How long will it take to run on an input size of 16?
 - **Let $T(n) = c \cdot n^2$**
 - $T(n)$ refers to the running time (of algorithm A) on input size n
 - Now, plug in the given data, and **find the value for c !**
 - $T(4) = c \cdot 4^2 = 10$ milliseconds
 - Therefore, **$c = 10/16$ milliseconds**
 - Now, answer the question by using c and solving $T(16)$
 - **$T(16) = c \cdot 16^2 = (10/16) \cdot 16^2 = 160$ milliseconds**

Big-O Notation – Practical Problems

- Practical Problems that can be solved utilizing order notation:
 - Example 2: Algorithm A runs in **$O(\log_2 n)$** time
 - For an input size of 16, the running time is 28 milliseconds
 - How long will it take to run on an input size of 64?
 - **Let $T(n) = c \cdot \log_2 n$**
 - Now, plug in the given data, and **find the value for c!**
 - $T(16) = c \cdot \log_2 16 = 10$ milliseconds
 - $c \cdot 4 = 28$ milliseconds
 - Therefore, **$c = 7$ milliseconds**
 - Now, answer the question by using c and solving $T(64)$
 - **$T(64) = c \cdot \log_2 64 = 7 \cdot \log_2 64 = 7 \cdot 6 = 42$ milliseconds**

More Algorithm Analysis

Kumkum Saxena

Big-O Notation

■ What is Big O?

■ Big O comes from Big-O Notation

- In C.S., we want to know how efficient an algorithm is...how “fast” it is
- More specifically...we want to know **how the performance of an algorithm responds to changes in problem size**
- The goal is to provide a *qualitative* insight on the # of operations for a problem size of n elements.
- And this total # of operations can be described with a mathematical expression in terms of n .
 - This expression is known as Big-O

More Algorithm Analysis

- Examples of Analyzing Code:
 - We now go over many examples of code fragments
 - Each of these functions will be analyzed for their runtime in terms of the variable n
 - Utilizing the idea of Big-O,
 - determine the Big-O running time of each

More Algorithm Analysis

■ Example 1:

- Determine the Big O running time of the following code fragment:

```
for (k = 1; k <= n/2; k++) {  
    sum = sum + 5;  
}  
for (j = 1; j <= n*n; j++) {  
    delta = delta + 1;  
}
```

More Algorithm Analysis

■ Example 1:

■ So look at what's going on in the code:

- We care about the total number of REPETITIVE operations.
 - Remember, we said we care about the running time for LARGE values of n
 - So in a for loop with n as part of the comparison value determining when to stop `for (k=1; k<= $n$ /2; k++)`
 - Whatever is INSIDE that loop will be executed a LOT of times
 - So we examine the code within this loop and see how many operations we find
 - When we say operations, we're referring to mathematical operations such as +, -, *, /, etc.

More Algorithm Analysis

■ Example 1:

■ So look at what's going on in the code:

- The number of operations executed by these loops is the sum of the individual loop operations.
- We have 2 loops,
 - The first loop runs $n/2$ times
 - Each iteration of the first loop results in one operation
 - The + operation in: `sum = sum + 5;`
 - So there are $n/2$ operations in the first loop
 - The second loop runs n^2 times
 - Each iteration of the second loop results in one operation
 - The + operation in: `delta = delta + 1;`
 - So there are n^2 operations in the second loop.

More Algorithm Analysis

■ Example 1:

■ So look at what's going on in the code:

- The number of operations executed by these loops is the sum of the individual loop operations.
- The first loop has $n/2$ operations
- The second loop has n^2 operations
- They are NOT nested loops.
 - One loop executes AFTER the other completely finishes
- So we simply ADD their operations
- The total number of operations would be $n/2 + n^2$
- In Big-O terms, we can express the number of operations as $O(n^2)$

More Algorithm Analysis

■ Example 2:

- Determine the Big O running time of the following code fragment:

```
int func1(int n) {  
    int i, j, x = 0;  
    for (i = 1; i <= n; i++) {  
        for (j = 1; j <= n; j++) {  
            x++;  
        }  
    }  
    return x;  
}
```


More Algorithm Analysis

■ Example 2:

■ So look at what's going on in the code:

- We care about the total number of REPETITIVE operations
- We have two loops
 - AND they are NESTED loops
- The outer loop runs n times
 - From $i = 1$ up through n
 - How many operations are performed at each iteration?
 - Answer is coming...
- The inner loop runs n times
 - From $j = 1$ up through n
 - And only one operation ($x++$) is performed at each iteration

More Algorithm Analysis

■ Example 2:

■ So look at what's going on in the code:

■ Let's look at a couple of iterations of the OUTER loop:

- When $i = 1$, what happens?
 - The inner loop runs n times
 - Resulting in n operations from the inner loop
- Then, i gets incremented and it becomes equal to 2
- When $i = 2$, what happens?
 - Again, the inner loop runs n times
 - Again resulting in n operations from the inner loop

■ We notice the following:

- For EACH iteration of the OUTER loop,
- The INNER loop runs n times
 - Resulting in n operations

More Algorithm Analysis

■ Example 2:

- So look at what's going on in the code:
 - And how many times does the outer loop run?
 - n times
 - So the outer loop runs n times
 - And for each of those n times, the inner loop also runs n times
 - Resulting in n operations
 - So we have n operations per iteration of OUTER loop
 - And outer loop runs n times
 - Finally, we have $n \times n$ as the number of operations
 - We approximate the running time as $O(n^2)$

More Algorithm Analysis

■ Example 3:

- Determine the Big O running time of the following code fragment:

```
int func3(int n) {  
    int i, x = 0;  
    for (i = 1; i <= n; i++)  
        x++;  
    for (i = 1; i <= n; i++)  
        x++;  
    return x;  
}
```

More Algorithm Analysis

■ Example 3:

■ So look at what's going on in the code:

- We care about the total number of REPETITIVE operations
- We have two loops
 - They are NOT nested loops
- The first loop runs n times
 - From $i = 1$ up through n
 - only one operation ($x++$) is performed at each iteration
- How many times does the second loop run?
 - Notice that i is indeed reset to 1 at the beginning of the loop
 - Thus, the second loop runs n times, from $i = 1$ up through n
 - And only one operation ($x++$) is performed at each iteration

More Algorithm Analysis

■ Example 3:

- So look at what's going on in the code:
 - Again, the loops are NOT nested
 - So they execute sequentially (one after the other)
- Therefore:
 - Our total runtime is on the order of $n+n$
 - Which of course equals $2n$
- Now, in Big O notation
 - We approximate the running time as $O(n)$

More Algorithm Analysis

■ Example 4:

- Determine the Big O running time of the following code fragment:

```
int func4(int n) {  
    while (n > 0) {  
        printf("%d", n%2);  
        n = n/2;  
    }  
}
```

More Algorithm Analysis

■ Example 4:

- So look at what's going on in the code:
 - We have one while loop
 - You can't just look at this loop and say it iterates n times or $n/2$ times
 - Rather, it continues to execute as long as n is greater than 0
 - The question is: **how many iterations will that be?**
 - Within the while loop
 - The last line of code divides the input, n , by 2
 - So n is halved at each iteration of the while loop
 - If you remember, we said this ends up running in $\log n$ time
 - Now let's look at how this works

More Algorithm Analysis

■ Example 4:

■ So look at what's going on in the code:

- For the ease of the analysis, we make a new variable
 - originalN:
 - originalN refers to the value originally stored in the input, n
 - So if n started at 100, originalN will be equal to 100
- The first time through the loop
 - n gets set to $\text{originalN}/2$
 - If the original n was 100, after one iteration n would be $100/2$
- The second time through the loop
 - n gets set to $\text{originalN}/4$
- The third time through the loop
 - n gets set to $\text{originalN}/8$

Notice:

After **three** iterations, n gets set to $\text{originalN}/2^3$

More Algorithm Analysis

■ Example 4:

■ So look at what's going on in the code:

- In general, after k iterations
 - n gets set to $\text{originalN}/2^k$
- The algorithm ends when $\text{originalN}/2^k = 1$, approximately
- We now solve for k
- Why?
 - Because we want to find the **total # of iterations**
- Multiplying both sides by 2^k , we get $\text{originalN} = 2^k$
- Now, using the definition of logs, we solve for k
 - $k = \log \text{originalN}$
- So we approximate the running time as $O(\log n)$

More Algorithm Analysis

■ Example 5:

- Determine the Big O running time of the following code fragment:

```
int func5(int** array, int n)  {
    int i = 0, j = 0;
    while (i < n) {
        while (j < n && array[i][j] == 1)
            j++;
        i++;
    }
    return j;
}
```

More Algorithm Analysis

■ Example 5:

- So look at what's going on in the code:
 - At first glance, we see two NESTED loops
 - This can often indicate an $O(n^2)$ algorithm
 - But we need to look closer to confirm
 - Focus on what's going on with i and j

```
int func5(int** array, int n) {  
    int i = 0, j = 0;  
    while (i < n) {  
        while (j < n && array[i][j] == 1)  
            j++;  
        i++;  
    }  
}
```

More Algorithm Analysis

■ Example 5:

- So look at what's going on in the code:
 - Focus on what's going on with i and j
 - i and j clearly increase (from the j++ and i++)
 - BUT, they never decrease
 - AND, neither ever gets reset to 0

```
int func5(int** array, int n) {  
    int i = 0, j = 0;  
    while (i < n) {  
        while (j < n && array[i][j] == 1)  
            j++;  
        i++;  
    }  
}
```

More Algorithm Analysis

■ Example 5:

■ So look at what's going on in the code:

- And the OUTER while loop ends once i gets to n
- So, what does this mean?
 - The statement i++ can never run more than n times
 - And the statement j++ can never run more than n times

```
int func5(int** array, int n) {  
    int i = 0, j = 0;  
    while (i < n) {  
        while (j < n && array[i][j] == 1)  
            j++;  
        i++;  
    }  
}
```

More Algorithm Analysis

■ Example 5:

■ So look at what's going on in the code:

- The MOST number of times these two statements can run (combined) is $2n$ times
- So we approximate the running time as $O(n)$

```
int func5(int** array, int n) {  
    int i = 0, j = 0;  
    while (i < n) {  
        while (j < n && array[i][j] == 1)  
            j++;  
        i++;  
    }  
}
```

More Algorithm Analysis

■ Example 6:

- Determine the Big O running time of the following code fragment:
 - What's the one big difference here???

```
int func6(int** array, int n) {
    int i = 0, j;
    while (i < n) {
        j = 0;
        while (j < n && array[i][j] == 1)
            j++;
        i++;
    }
    return j;
}
```


More Algorithm Analysis

■ Example 6:

- So look at what's going on in the code:
 - The difference is that we RESET j to 0 at the beginning of the OUTER while loop

```
int func6(int** array, int n) {
    int i = 0, j;
    while (i < n) {
        j = 0;
        while (j < n && array[i][j] == 1)
            j++;
        i++;
    }
    return j;
}
```

More Algorithm Analysis

■ Example 6:

■ So look at what's going on in the code:

- The difference is that we RESET j to 0 at the beginning of the OUTER while loop
- How does that change things?
 - Now j can iterate from 0 to n for EACH iteration of the OUTER while loop
 - For each value of i
 - This is similar to the 2nd example shown
- So we approximate the running time as $O(n^2)$

More Algorithm Analysis

■ Example 7:

- Determine the Big O running time of the following code fragment:

```
int func7(int A[], int sizeA, int B[], int sizeB) {  
    int i, j;  
    for (i = 0; i < sizeA; i++)  
        for (j = 0; j < sizeB; j++)  
            if (A[i] == B[j])  
                return 1;  
    return 0;  
}
```

More Algorithm Analysis

■ Example 7:

■ So look at what's going on in the code:

- First notice that the runtime here is NOT in terms of n
- It will be in terms of sizeA and sizeB
- And this is also just like Example 2
- The outer loop runs sizeA times
- For EACH of those times,
 - The inner loop runs sizeB times
- So this algorithm runs $\text{sizeA} * \text{sizeB}$ times
- We approximate the running time as $O(\text{sizeA} * \text{sizeB})$

More Algorithm Analysis

■ Example 8:

- Determine the Big O running time of the following code fragment:

```
int func8(int A[], int sizeA, int B[], int sizeB) {  
    int i, j;  
    for (i = 0; i < sizeA; i++) {  
        if (binSearch(B, sizeB, A[i]))  
            return 1;  
    }  
    return 0;  
}
```

More Algorithm Analysis

■ Example 8:

■ So look at what's going on in the code:

- Note: we see that we are calling the function `binSearch`
- As discussed previously, a single binary search runs in $O(\log n)$ time
 - where n represents the number of items within which you are searching

■ Examining the for loop:

- The for loop will execute `sizeA` times
- For EACH iteration of this loop
 - a binary search will be run
- We approximate the running time as $O(\text{sizeA} * \log(\text{sizeB}))$

And More Algorithm Analysis

Kumkum Saxena

And More Algorithm Analysis

■ Examples of Analyzing Code:

- Last time we went over examples of analyzing code
 - We did this in a somewhat naïve manner
 - Just analyzed the code and tried to “trace” what was going on
- This Lecture:
 - We will do this in a more structured fashion
 - We mentioned that summations are a tool for you to help coming up with a running time of iterative algorithms
 - Today we will look at some of those same code fragments, as well as others, and show you how to use summations to find the Big-O running time

More Algorithm Analysis

■ Example 1:

- Determine the Big O running time of the following code fragment:
 - We have two for loops
 - They are NOT nested
 - The first runs from $k = 1$ up to (and including) $n/2$
 - The second runs from $j = 1$ up to (and including) n^2

```
for (k = 1; k <= n/2; k++) {  
    sum = sum + 5;  
}  
for (j = 1; j <= n*n; j++) {  
    delta = delta + 1;  
}
```

More Algorithm Analysis

■ Example 1:

- Determine the Big O running time of the following code fragment:
 - Here's how we can express the number of operations in the form of a summation:

$$\sum_{k=1}^{n/2} 1 + \sum_{j=1}^{n^2} 1$$

The constant value, 1, inside each summation refers to the one, and only, operation in each for loop.

```
for (k = 1; k <= n/2; k++) {  
    sum = sum + 5;  
}  
for (j = 1; j <= n*n; j++) {  
    delta = delta + 1;  
}
```

Now you simply
solve the summation!

More Algorithm Analysis

■ Example 1:

- Determine the Big O running time of the following code fragment:

- Here's how we can express the number of operations in the form of a summation:

$$\sum_{k=1}^{n/2} 1 + \sum_{j=1}^{n^2} 1$$

You use the formula: $\sum_{i=1}^n k = k * n$

$$\sum_{k=1}^{n/2} 1 + \sum_{j=1}^{n^2} 1 = \frac{n}{2} + n^2$$

- This is a **CLOSED FORM** solution of the summation
- So we approximate the running time as $O(n^2)$

More Algorithm Analysis

■ Example 2:

- Determine the Big O running time of the following code fragment:
 - Here we again have two for loops
 - But this time they are nested

```
int func2(int n) {  
    int i, j, x = 0;  
    for (i = 1; i <= n; i++) {  
        for (j = 1; j <= n; j++) {  
            x++;  
        }  
    }  
    return x;  
}
```

More Algorithm Analysis

■ Example 2:

- Determine the Big O running time of the following code fragment:
 - Here we again have two for loops
 - But this time they are nested
 - The outer loop runs from $i = 1$ up to (and including) n
 - The inner loop runs from $j = 1$ up to (and including) n
 - The sole (only) operation is a “ $x++$ ” within the inner loop

More Algorithm Analysis

■ Example 2:

- Determine the Big O running time of the following code fragment:

- We express the number of operations in the form of a summation and then we solve that summation:

$$\sum_{i=1}^n \sum_{j=1}^n 1$$

You use the formula: $\sum_{i=1}^n k = k * n$

$$\sum_{i=1}^n \sum_{j=1}^n 1 = \sum_{i=1}^n n = n^2$$

All we did is apply the above formula twice.

- This is a **CLOSED FORM** solution of the summation
- So we approximate the running time as $O(n^2)$

More Algorithm Analysis

■ Example 3:

- Determine the Big O running time of the following code fragment:
 - Here we again have two for loops
 - And they are nested. So is this $O(n^2)$?

```
int func3(int n) {  
    sum = 0;  
    for (i = 0; i < n; i++) {  
        for (j = 0; j < n * n; j++) {  
            sum++;  
        }  
    }  
}
```

More Algorithm Analysis

■ Example 3:

- Determine the Big O running time of the following code fragment:
 - Here we again have two for loops
 - And they are nested. So is this $O(n^2)$?
 - The outer loop runs from $i = 0$ up to (and not including) n
 - The inner loop runs from $j = 0$ up to (and not including) n^2
 - The sole (only) operation is a “sum++” within the inner loop

More Algorithm Analysis

■ Example 3:

- Determine the Big O running time of the following code fragment:

- We express the number of operations in the form of a summation and then we solve that summation:

$$\sum_{i=0}^{n-1} \sum_{j=0}^{n^2-1} 1$$

You use the formula: $\sum_{i=1}^n k = k * n$

$$\sum_{i=0}^{n-1} \sum_{j=0}^{n^2-1} 1 = \sum_{i=0}^{n-1} n^2 = n^2 \sum_{i=0}^{n-1} 1 = n^3$$

All we did is apply the above formula twice.

- This is a **CLOSED FORM** solution of the summation
- So we approximate the running time as $O(n^3)$

More Algorithm Analysis

■ Example 4:

- Write a summation that describes the number of multiplication operations in this code fragment:
 - Here we again have two for loops
 - Pay attention to the limits (bounds) of the for loop

```
int func3(int n) {  
    bigNumber = 0;  
    for (i = 100; i <= 2n; i++) {  
        for (j = 1; j < n * n; j++) {  
            bigNumber += i*n + j*n;  
        }  
    }  
}
```

More Algorithm Analysis

■ Example 4:

- Write a summation that describes the number of multiplication operations in this code fragment:
 - Here we again have two for loops
 - Pay attention to the limits (bounds) of the for loop
 - The outer loop runs from $i = 100$ up to (and including) $2n$
 - The inner loop runs from $j = 1$ up to (and not including) n^2
 - Now examine the number of **multiplications**
 - Because this problem specifically said to “describe the number of multiplication operations, we do not care about ANY of the other operations
 - `bigNumber += i*n + j*n;`
 - There are TWO multiplication operations in this statement

More Algorithm Analysis

■ Example 4:

- Write a summation that describes the number of multiplication operations in this code fragment:

- We express the number of multiplications in the form of a summation and then we solve that summation:

$$\sum_{i=100}^{2n} \sum_{j=1}^{n^2-1} 2$$

$$\sum_{i=100}^{2n} \sum_{j=1}^{n^2-1} 2 = \sum_{i=100}^{2n} 2(n^2 - 1) = 2(n^2 - 1) \sum_{i=100}^{2n} 1 = 2(n^2 - 1)(2n - 99)$$

- This is a **CLOSED FORM** solution of the summation
- Shows the number of multiplications